

Subscriptions Flow AI

Team Keepify

Namsang Subba Liang Chu Joseph Skinner
Isaac Traina Jonathan Vu

Lightning Lab Solutions

John Wurzbacher

Faculty Coach

Samuel Malachowsky

Project Overview

Intro

Shopify is a global e-commerce platform where over 5 million merchants choose to sell their products in 6 million unique stores to over 1 billion current active shoppers. The platform features a robust experience for merchants to handle the most crucial aspects of their online retail businesses including product display, payment processing, customer information handling and so much more. Shopify even provides a rich array of APIs that allows developers to extend this ecosystem to provide additional features for shoppers and merchants alike. Subscription management apps are some of the most ubiquitous use cases of such APIs, allowing merchants to capitalize on stable and consistent revenue while shoppers gain the benefit of frictionless purchases. However, many 3rd party applications do not directly address customer cancellations, otherwise known as churn, nor do they attempt to leverage AI to optimize solutions against it. As such, merchants remain unaware of the health and status of their subscriptions-based business and do not have the insights necessary to make informed decisions regarding the direction of their selling plans.

Purpose

The main goal of this project is to create an app that adheres to the following description:

Subscriptions Flow AI is a Shopify application that tackles subscription churn - the constant loss of subscribers due to cancellation or expiration. This is an issue that reduces merchants' predictable revenue, and in turn can make or break any of the 5.6 million active Shopify businesses. Subscriptions Flow AI supports merchants by allowing them to intervene whenever a customer is attempting to cancel. Before a customer has the chance to cancel, the ChurnGuard AI model will evaluate the churn risk for all customers. Based on this analysis, merchants will have the ability to initiate upsell methods to attempt to retain customers, such as offering them a discount. Store owners can also create custom discount offers based on delivery and subscription intervals. Depending on merchant needs, the application can be extended to support increased automation - streamlining their experience by suggesting personalized strategies for customer retention.

Scope

What is in scope:

1. Shopper Interface
 - a. The app will show shoppers the type of discount available to them when choosing a subscription delivery and payment plan
 - b. Shoppers will be prompted with retention strategies upon cancellation
2. Admin Interface
 - a. The app will include interfaces for admins to control various aspects of the app that include the following
 - i. Creating subscription plans
 - ii. Setting up subscriptions for users
 - b. Admins will be able to view analytics pertaining to the churn scores and factors of their products and customers
3. Cancellation Interventions
 - a. The app will be able to offer rewards to users in the form of discounts for not canceling their subscriptions
4. AI
 - a. The app will use machine learning and or AI features to determine churn scores
 - b. AI will be integrated into the app as a core feature that users are aware of
5. Churn Score Explanations
 - a. The app will provide reasons to the admin for why each customer has a certain churn score
6. The app will adhere to Shopify UI and App standards / regulations
 - a. The use of Polaris UI
 - b. Shopify security / webhooks / content redaction requests
7. The app will have multiple tiers that offer different levels of control over the aforementioned features*

* Denotes a planned feature that did not make it into the final product due to time constraints

What is not in scope:

1. The app will not remake any of Shopify's API features
 - a. Anything that can be done by the Shopify API will be required to use the API
2. The app will not handle global business any differently than Shopify does
3. The app will not include an AI chatbot
4. The app will not remind users of their unused subscriptions / interact with users who are at risk of churn unprompted
5. Bonus features mentioned in the document

Core Features

The following categories represent the core features that SubscriptionsFlowAI implements in order to adhere to the requirements of the project sponsor.

Subscriptions Management

These features pertain to the ability for a merchant to sell their regular products as subscription contracts. Shoppers can thus be automatically billed for recurring delivery of items and services, an absolute necessity for merchants to capture consistent and reliable revenue.

Customizable Selling Plan Groups

Using this feature merchants are able to create, update and delete selling plan groups which are Shopify data entities that represent the parameters by which a product can be bought through a subscription contract. Such parameters include active start and end dates, product placement, recurring billing/delivery frequency, prepaid billing/delivery frequency, and discounts both percent and flat rates. Each of these fields may be edited after creation and such changes are automatically applied to any future subscription. After a merchant creates a selling plan group so long as they place the Shopper Widget into their frontend store then a shopper will be able to checkout each product as a subscription.

Shopper Widget

The Shopper Widget is the set of UI components that a shopper will see when they attempt to checkout a product that is included in any valid selling plan group. The merchant is able to modify the look and feel of the widget when they are creating a selling plan group. Depending on which template of plan the merchant chooses the widget will change accordingly to display relevant information, for example if the template includes both prepaid and recurring billing options then the widget will show that - otherwise it will not. Primarily, the merchant has the ability to change the color scheme of the widget to seamlessly match the theme of their store without having to go back and forth between their admin app and the shopper frontend store customizer screen.

Generate Checkout Link

Often, shoppers who are having issues with using a Shopify storefront will contact the merchant directly via email or phone in order to place their orders. Shopify already supports this feature when it comes to creating static, one time orders however it does not provide a way to do this for subscription contracts. SubscriptionsFlowAI gives merchants the ability to generate shareable checkout links with shoppers via email that will allow them to directly purchase a subscription based product while bypassing the storefront. In this way, merchants are able to capture subscription sales even if, for whatever reason, the shopper is unable to checkout using the regular storefront.

Analytics

SubscriptionsFlowAI provides a rich set of features in supporting merchants to sell their products as subscriptions. This core feature would be incomplete without providing the ability for merchants to view relevant metrics that give insights into the health and status of their business. Specifically, the app collects and shows merchants the top 10 products that are being bought as subscriptions, a graph of subscription contracts created over time, the total revenue generated from subscription contracts and the total amount of active subscriptions their store currently has. Merchants can use this information to

decide whether or not they want to create new selling plans groups, delete old ones or modify existing plans in order to increase revenue.

Artificial Intelligence and Machine Learning

SubscriptionsFlowAI is dedicated to leveraging artificial intelligence and machine learning in intentful ways to enhance the experience of merchants using Shopify. The following features aim only to provide more value to merchants and make active efforts to involve them more in the decision making processes of their business.

Churn Score

Churn is a metric that tells merchants how often customers are cancelling their subscription contracts. SubscriptionsFlowAI intelligently measures a churn score for each customer, otherwise, the likelihood that a customer may cancel any of their contracts. The higher the score, the greater the chance of cancellation. The app calculates this using a linear regression model that has been trained on over 5000 rows of general shopper and subscription based data. Merchants are able to view the churn scores for each customer in a table that allows filtering and sorting by churn score as well. Such actions that may affect a customer's score include: Creating a new address, cancelling one of their plans, entering a new subscription contract, buying other products, being a customer for an extended period of time etc...

Churn Explanation

SubscriptionsFlowAI uses the Google Gemini LLM in order to condense all of the factors that affect a customer's churn score into a simple three bullet point summary. From the same page where they can view their store's subscribers, merchants are able to select a customer to see why their score is what it is. The impact of this feature is that merchants do not need to guess not only why a score is what it is but also why a score may have changed. Given a summary of explanations, a merchant is able to make more informed decisions about their subscriptions-based business and overall understand their shoppers' behavior.

Remedy offers

SubscriptionsFlowAI works to prevent churn by offering customers remedy options prior to cancellation. When a customer attempts to cancel their plan they are met with a feedback form that captures the reason for them leaving. If a merchant has enabled this feature, once a customer has finished providing feedback, they will be offered a discount based on parameters set by the merchant. In the current iteration of the app, customers are offered a max discount set by the merchant in the settings. However, the functionality of the AI deciding where in the range the discount should fall exists on the churn explanation page and simply needs to be linked to the canceling process.

Built for Shopify

Third party apps on the Shopify app store may qualify for the "Built for Shopify" badge by adhering to a set of requirements that attest to the apps overall quality. Receiving this badge is an honor for any application and proves to users that the app has been thoroughly verified by the Shopify community. SubscriptionsFlowAI aims to follow the guidelines below in order to strive for this badge.

Polaris UI

The Shopify app development community is encouraged to follow the guidelines of the Polaris UI framework. At the core of this framework is the principle of best practices and by using Polaris web components developers are able to easily meet these standards. Specifically, the most important concepts are consistent component spacing, consistent and reusable components, familiar user experience, adaptable components, and accessibility. When developers commit to choosing Polaris UI components first, they are able to automatically fulfill such requirements.

Constraints

Shopify App Approval Requirements

In order for an app to be listed on the Shopify app store it must adequately meet the following requirements.

Functionality

SubscriptionsFlowAI must not be similar to any app that the sponsor's organization has already created. This was not an issue since our app is unique amongst the sponsor's ecosystem. The app must also not enable any malicious behavior by either merchant or shopper.

Data Protection Webhooks

When a customer elects to delete their account from a merchant's store, any app that was using or storing that data must delete it. Shopify GraphQL data is automatically deleted. However, our app does store Shopper data in our AWS RDS instance when collecting customer feedback. This data is deleted accordingly when necessary. Shoppers are also justly able to request and receive such data at will.

Cloud Costs

SubscriptionsFlowAI must be supported by cloud-based infrastructure in order to provide crucial product and development environment functionality. Specifically, AWS is used to host both environment's databases to ensure that developers can freely run and test the application without the possibility of affecting existing data in the deployed instance. AWS is also used to host the application using EC2 which allows merchants to install and access the app from the Shopify app store. The project is constrained to using only free/and or the cheapest available tiers that support the aforementioned functionality. If the project were to continue, the next dev team may consider increasing database storage to support more users or more features that require more data.

AI Costs

Artificial intelligence is paramount to providing maximum value for merchants while they are using SubscriptionsFlowAI. In its current state, the app uses artificial intelligence and machine learning to generate and explain churn scores for each subscriber. These systems are limited to the most basic tiers of cloud infrastructure in order to support their functionality. For example, we currently need to retrain the linear regression model manually rather than automatically based on arbitrary triggers due to the cost of hosting a lambda function to do this. Similarly, we are using a free version of Google Gemini's LLM model to digest and display explanations for a shopper's churn score in order to reduce costs.

This has manifested in the need for a button to manually generate churn explanations as to not exceed the amount of tokens allotted for the free tier model. Future iterations of the project may consider upgrading to a paid tier that can support multiple users, including this cost in higher payment tiers of SubscriptionsFlowAI itself.

App Development

SubscriptionsFlowAI is first and foremost a Shopify application. In order to seamlessly integrate our app into the Shopify platform, we often made decisions based around this constraint. Our app was initially created using Shopify's own developer command line interface which provides a scaffolded React + Router application and some template code to get developers started. Shopify also happens to provide the most convenient access to their own data essential for creating apps for its platform via GraphQL - a query language designed around vertices and nodes instead of tables and keys. Additionally, apps integrated directly into the Shopify admin portal are strongly advised to use the Polaris UI framework in order to create a user experience that is consistent with the rest of the platform. Thus, our team was greatly constrained technologically by not being able to directly make decisions about which tools we wanted to use. This was ultimately to the team's benefit as it allowed us to design and implement an application that would be familiar to merchants, shoppers and other developers of Shopify apps alike.

Process Metrics

Some of the process metrics that the team kept track of each sprint include earned value and sprint velocity. We allowed these metrics to constrain the development of the product to ensure that deliverables and artifacts were able to be completed within our projected timeline. Specifically, during sprint planning, we purposefully left out certain tickets if adding them meant our potential velocity would be much greater than our average.

Development Process

Our team used an Agile Scrum methodology to guide the development of the system. We worked in bi-weekly sprints, with sprint planning at the beginning of each sprint and a sprint review/demo at the end. This allowed the team to regularly evaluate progress, receive feedback, and adjust priorities based on project needs.

The process was approved by the sponsor and was not strictly mandated. The team selected Scrum because it provided a structured but flexible approach for managing development, tracking progress, and responding to sponsor feedback throughout the project. Scrum ceremonies such as sprint planning, standups, sprint reviews, and retrospectives helped the team stay organized, transparent, and accountable.

Communication with the sponsor was built directly into the process. At the beginning of the project, sponsor communication was handled through email, and later the team migrated communication to Discord for faster and more consistent interaction. The Sponsor Liaison was responsible for maintaining regular communication with the sponsor. Each week before the meeting, the Sponsor Liaison emailed the meeting agenda to both the Sponsor and the Coach. The agenda included discussion topics, project updates, and a standing Zoom link in case any participant could not attend in person. Sponsor meetings also served as sprint review opportunities, where the team demonstrated

progress, gathered feedback, and confirmed that development remained aligned with sponsor expectations.

The team identified several key roles to support the project: Scrum Lead, Sponsor Liaison, Project Manager, AI Tech Lead, and Cloud Tech Lead. These roles were filled based on each team member's strengths, interests, and technical background. The Scrum Lead was responsible for facilitating Scrum ceremonies and keeping the team aligned with the Agile process. The Sponsor Liaison managed communication with the sponsor and coach. The Project Manager helped track deadlines, deliverables, Jira tasks, and overall project progress. The AI Tech Lead guided AI-related design and implementation decisions, while the Cloud Tech Lead oversaw cloud infrastructure and deployment responsibilities. Overall, the team's Scrum-based process provided a clear structure for planning, communication, development, review, and improvement throughout the project.

Project Schedule: Planned and Actual

The team developed a schedule through a combination of discussion with the sponsor and analysis of deliverables on the Senior Project website. We had three sets of milestones: beginning of the project (9/25), the beginning of second semester (1/20), and the end of the project (4/30).

Milestones as of 9/25:

1. Housekeeping and Project Organization

Before the team can begin coding, there should be a unanimously solid foundation of knowledge between team members and the project sponsor. This includes creating necessary diagrams and documentation of the project domain, scope and synopsis. The team must also set up their project management and version control tools as well as agree on implementation tools. After review from the project sponsor the team will be able to begin setting up for developing the app.

2. Tickets / Epics Scaffolding

Before we begin development we should have tickets and epics in our backlog that represent all of what needs to be done to support the initial MVP. These tickets do not need to be fully fleshed out - they just need to exist and be able to map 1:1 to the requirements doc.

3. UI Review and Mockup Approval

Our sponsor has provided us with UI mockups for his vision of the app. Until we map out the requirements with the UI we cannot know for sure if the UI fully supports all of the functionality. Additionally, we do not know how efficient the UI / UX is until we test it. The team will create mockups of the UI that reflect this analysis.

4. DBMS Plan

We need to plan out the schema for our data models. We also need to plan where and how we are going to host the database.

5. AI Plan

We need to plan out specifically how AI will be integrated into the app. This can include the following considerations

1. Where is the AI hosted / which model
2. Which features are impacted by AI

6. Initial MVP

The initial MVP will contain most if not all of the non-bonus requirements laid out by the project sponsor. This will be iteratively worked on throughout the semester in an Agile Scrum methodology. This may further be broken down into the following

1. Initial UI adherence
2. Backend API
3. Database management
4. App testing

The initial MVP is scheduled to be delivered on Tuesday November 25th 2025

Time permitting the following features may be added to the project scope and will expected in the MVP - otherwise they are planned to be included in future releases

1. Win back feature
2. Migration to SubscriptionsFlowAI from other similar apps

7. Project Retrospective

At the conclusion of the semester a retrospective will be held assessing the progress made during this term. Team members will submit relative documentation and hold a meeting with the project coach.

8. Feedback

Upon release of the initial MVP, there will be a free trial period for beta testers to use the app. Future development of the application will proceed according to this feedback.

9. Feature freeze - planned to occur tentatively near the end of March 2026.

10. The final product will be delivered 2-3 weeks after the planned feature freeze and will include the following hand off documents and deliverables

- Project handoff document
- Release of permissions to all dev environments
- Access to Jira board (if requested)
- Official handoff and continuation of development docs

Milestones as of 1/20:

1. Housekeeping and Project Organization - Completed
2. Ticket/Epic Scaffolding - Completed
3. UI Review and Mockup Approval - Completed
4. Initial MVP + App Deployment - In Progress
5. Project Retrospective - Completed
6. Onboard Technical Intern - In Progress
7. Feedback - Not Started
8. Feature Freeze - Not Reached
9. Code Freeze - Not Reached
10. Product Handoff - Not Started

Discrepancies:

- The “Feedback” milestone was not able to be reached by 11/25. This was an extremely ambitious date estimate, as we realized the deployment process was much more involved than expected. Additionally, the AI features were not developed enough to be tested in a meaningful way.
- Deployment needed to be added to the list, as it needed to be done alongside Initial MVP.
- The team opted in to bring on a technical intern. To help the intern get acquainted with the project smoothly, we needed to prepare a list of documentation, decide who should run check-in meetings and when, and come up with a list of introductory project tasks.
- Code freeze was left off the original list for some reason and was added here.

Adaptations:

- The team was still able to obtain feedback throughout the Spring semester, just not through a Shopify app listing. Though smaller in number of users, the feedback itself was more detailed than what a random user was likely to provide. Testing sessions ranged from 1-3 hours.

Milestones as of 4/30:

1. Initial MVP + App deployment - Completed
2. Shopify App Approval - Incomplete
3. Onboard Technical Intern - Completed
4. Feedback - Completed
5. Feature Freeze - Reached
6. Code Freeze - Reached
7. Product Handoff - In Progress

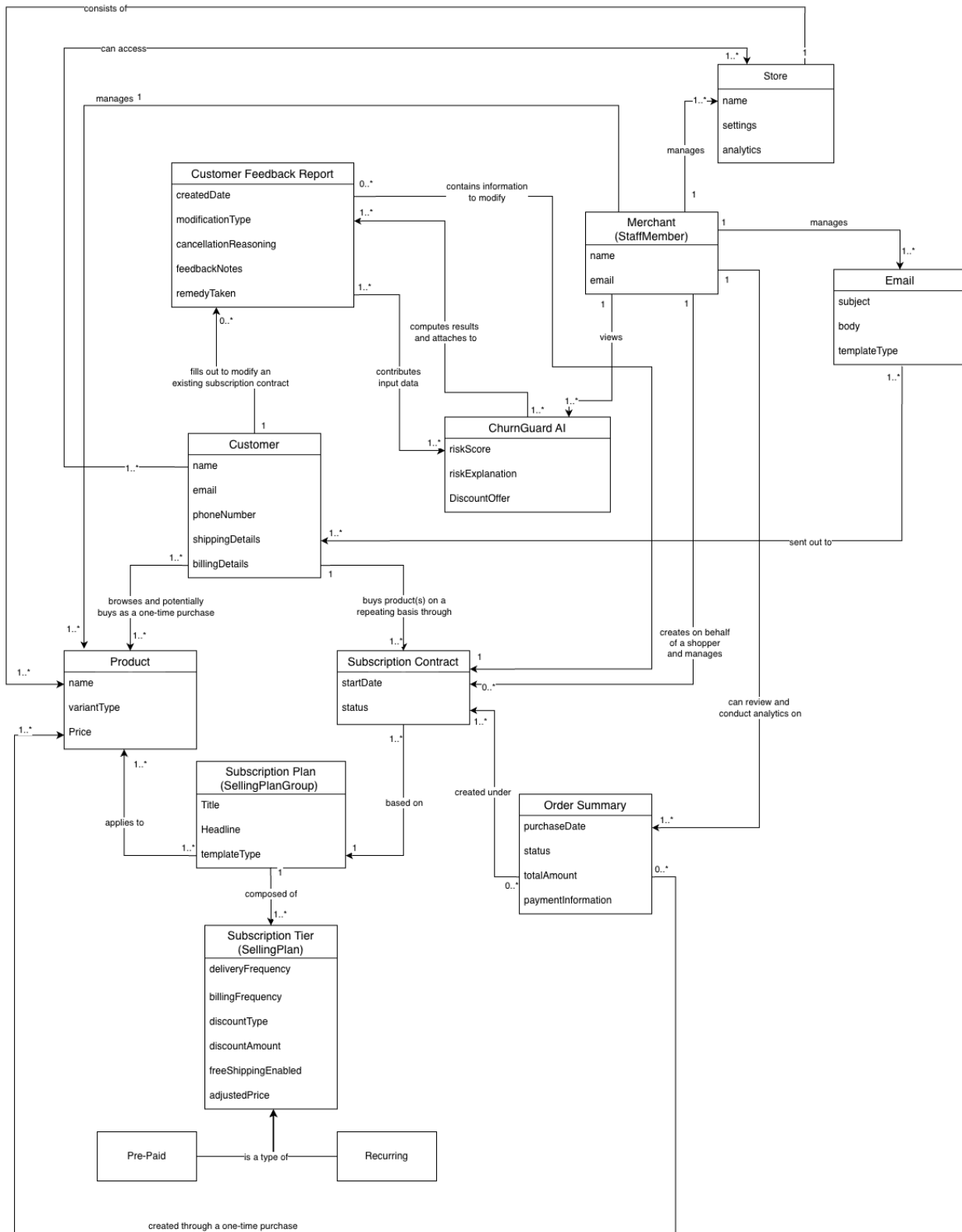
Discrepancies:

- We originally assumed approval of the Shopify app would occur alongside deployment and be finished by the time this report was written. Subscriptions Flow AI has been submitted for review since March 20th. However, Shopify still has yet to assign a reviewer to the app.

Adaptations:

- Nothing can be done to speed up app approval, but we have communicated with the sponsor's other development team to ensure they have everything they need when approval goes through.

Domain Model



The above diagram presents our finalized conceptual understanding of Subscriptions Flow AI. It illustrates the key entities and relationships within features like our core subscription management workflow, AI churn prediction model, customer feedback report system, and merchant configuration settings and store analytics.

Starting with the core subscription management workflow, our domain model illustrates how Merchants/StaffMembers and Shoppers/Customers go through a continuous, multi-phase lifecycle. The process begins with merchants configuring their Subscription Plans/Selling Plan Groups by assigning specific products, defining a title and headline, and selecting a pre-configured UI template (Minimalist, Discount, or Coupon). The next phase involves configuring the Subscription Tiers/Selling Plans for a Subscription Plan. With this phase, merchants can configure specific details such as billing and delivery frequencies, payment models (such as pre-paid vs recurring payment models), free shipping eligibility, and discount types (such as percentages vs fixed amount) that help automatically calculate the final unit price. After that, merchants can start offering subscription options directly onto their storefront through an extension widget. They can also fill out a separate form that allows them to manually generate a custom checkout web link of a subscription plan for a specific customer. Upon a successful checkout, an Order Summary is created detailing the purchase date, total amount spent, and Shopify-encrypted payment data. During this phase, it will also subsequently trigger the creation of a Subscription Contract to track the plan's active status and start date. Finally, the Customer Management phase is facilitated by a different widget that's manually added to the orders page by the merchant. This widget allows shoppers to independently view and manage their active subscriptions.

Moving onto the AI churn prediction models, our domain model showcases how Store Merchants will be able to obtain real-time reports on how likely a Shopper is going to cancel at least one of their subscriptions. In order to view this information, Merchants will have access to a subscribers page that showcases information like shopper's active subscription contracts as well as an analytics report from our ChurnGuard AI model. This model produces a risk score of how likely a shopper is from cancelling one of their subscription contracts. The higher the percentage, the more likely it is for the Shopper to cancel a subscription contract in the near future. This model also gives out a risk explanation on why it had come up with that score in the first place. Lastly, the model gives out a suggested discount offer to the Shopper to prevent them from cancelling any of their current subscription contracts. This type of discount is a one-time offer that can work alongside with the subscription base discount, if applicable.

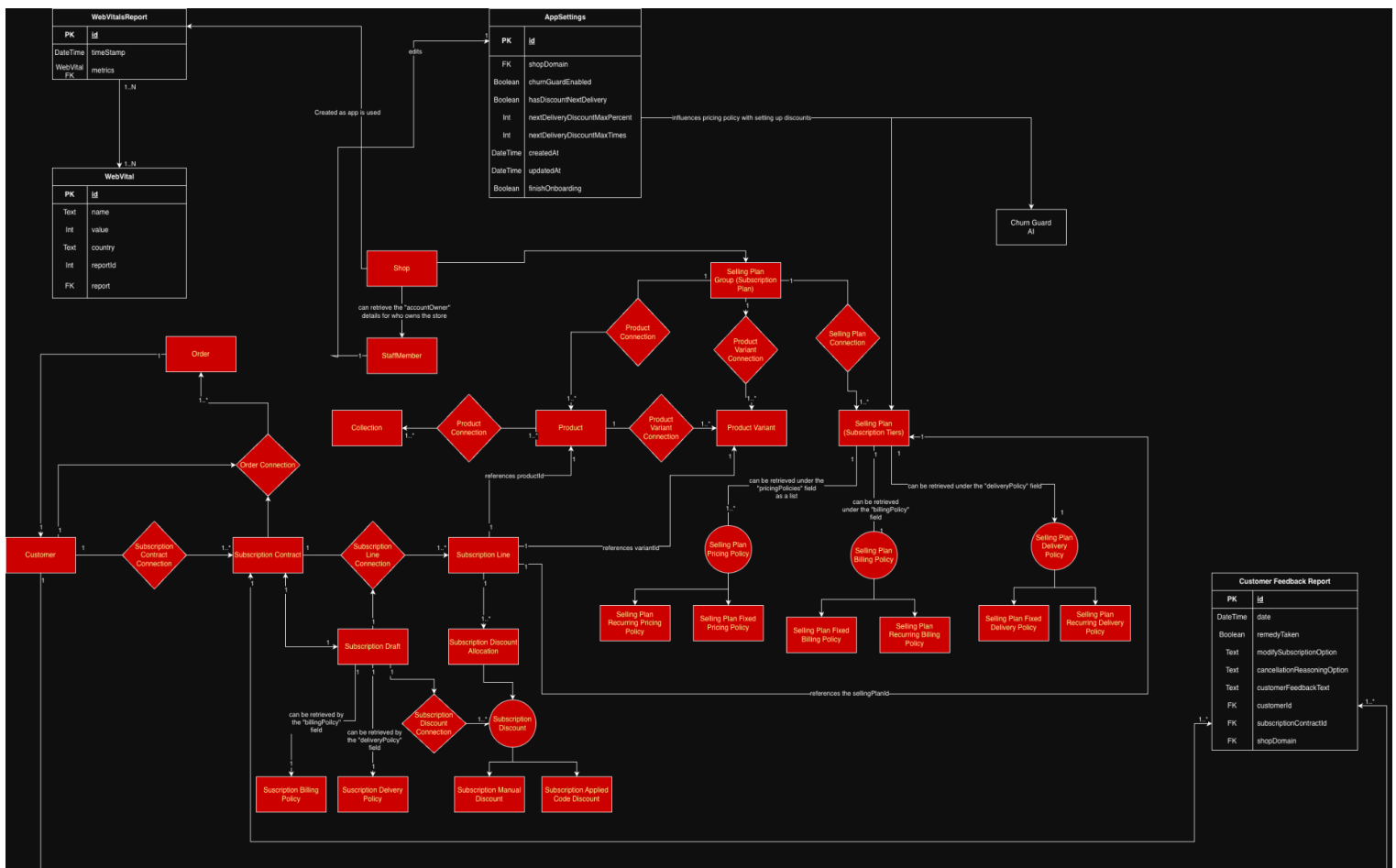
If shoppers want to modify their current subscription contracts in any way, then they can use our Customer Feedback Reporting System. This system is accessible through the subscription contract widget on the shopper's order page of the storefront. Each time a shopper wants to make a modification and alter the state of an active contract, they can proceed to press any 4 modification types: Skip the next delivery of their contract, pause their contract, swap their current product with a different variant, or cancel their contract. For cancelling a subscription contract specifically, the system redirects them to a form, stating the reasoning behind cancelling said subscription contract. They can choose between a couple of pre-configured options like: "It's Too Expensive", "Have too much of the product", "Found a better price elsewhere", or "Need to save money". Shoppers can also click "N/A" for no response or for when all options don't apply to them. For any of these situations, Shoppers can also expand upon their thoughts for cancelling by providing additional feedback notes. From here, if the shopper has filled out the initial cancellation form, they may be able to receive a one-time discount offer based on the ChurnGuard AI model as well as the Merchant's configuration settings. Ideally, the

Shopper Feedback Reports would eventually be reviewed by the Merchant. However, due to prioritizing other, more important MVP features, we were unable to implement this within our application, thus there is no current relationship seen within our domain model above.

Regarding merchant configurations, the domain model illustrates a comprehensive suite of in-app settings. These include toggles for enabling the ChurnGuard AI and one-time delivery discounts, as well as parameters defining the maximum allowable discount percentage and the frequency limits for retention offers. Additionally, our system contains a separate dedicated Store Analytics page that provides merchants with high-level metrics, such as their overall revenue and top selling products.

Finally, the domain model accounts for an Email entity, though its physical implementation was scoped out of the current development cycle. Initially, we planned on implementing a custom automated email system triggered by internal state changes like sending out emails when product inventory was low or when an order is confirmed or denied. However, further investigation revealed that Shopify themselves have their own automated system that inherently handles most of these transactional events. After consulting with our project sponsor, it was determined that the remaining edge-case events were not critical for the initial launch of the application. Thus, this feature was scoped out of the MVP to prioritize core functionality. For our domain model though, we still showcased the Email entity to reflect the system utilization of Shopify pre-configured email template types, subjects, and bodies.

Database Diagram

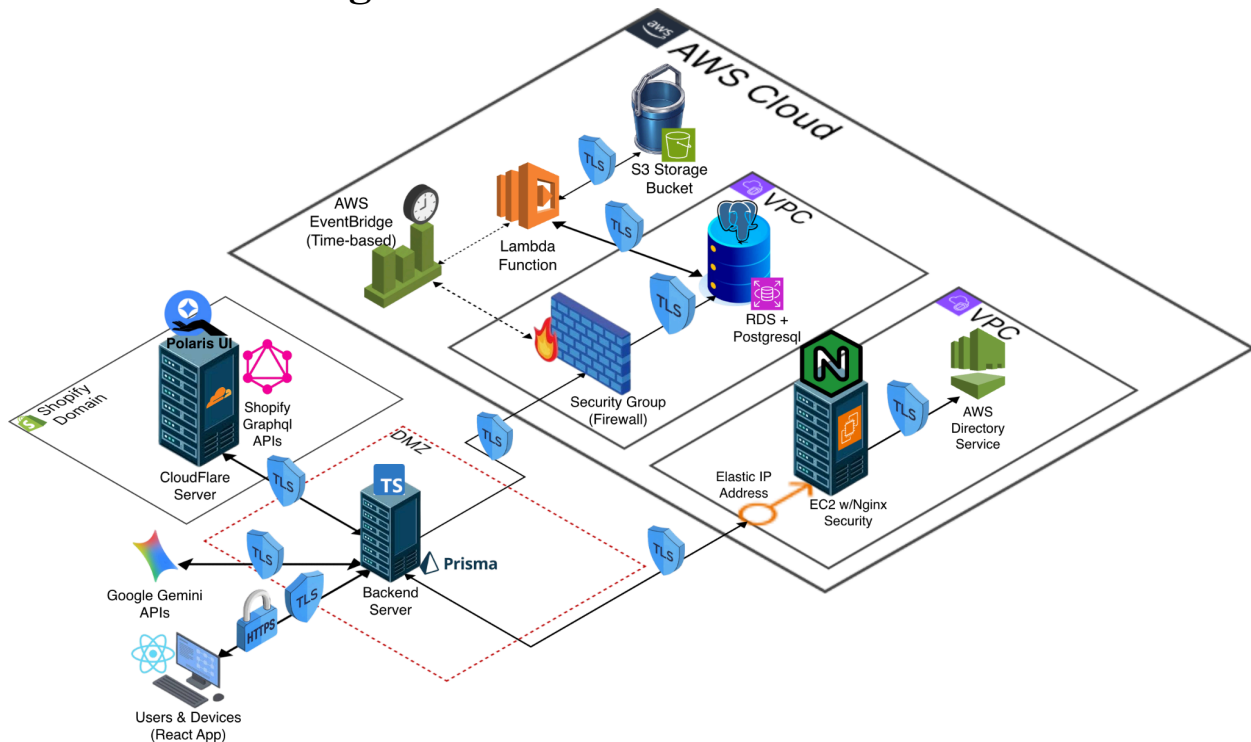


The diagram above maps out the finalized internal data structure of the Subscriptions Flow AI application. It highlights the elements Shopify handles for us, like objects, unions, and connections between objects. In addition to Shopify's data structures, we implemented our own custom, cloud-hosted data tables to manage unique data schemas and extend functionality that falls outside the scope of Shopify's native environment.

For all of the data that Shopify is able to handle, store, and gain access to, our database model above highlights them in red. The red rectangles represents the objects that we can query against including information about Shop and StaffMembers, Selling plan Groups, Selling Plans, Products, Collections of Products, Product Variants, Pricing, Billing, and Delivery policies, Orders, Customers, Subscription Contracts, Subscription Lines within Contracts. The red diamonds represent the connections to join together and return the data from one object to the other. This can be seen throughout the database model for example, the Selling Plan Group has direct connections to the products, their product variants, as well as the selling plans (or tiers). The red circles, which are the unions, highlight overlapping entities where a single conceptual object is represented through multiple data schemas. This can be seen throughout the database model as there's different types of Pricing, Billing, and Delivery policies.

For our own custom data tables, we designed them specifically to support three core operations: the customer feedback reporting system, in-app merchant configurations, and web vital telemetry for post-deployment monitoring. All of our custom data tables are hosted via the cloud. We did have some other custom data tables that were partially implemented or had conceptually thought of implementing for this application. Those data tables were specifically for our automated email feature as well as the ChurnGuard AI model. If we fully implemented our automated email feature, we would've had two data tables where one of them was storing the email templates we initially created and then the other being the actual emails that were scheduled to be sent out by a certain event or time-frame. However, due to the reasons mentioned in the domain model section where most of the emails that we wanted to automate were already handled by Shopify through their email webhooks, we decided to halt any further implementation. Thus it's not shown within our database model. Regarding the ChurnGuard AI model, we identified that a dedicated data table for historical report logs would be highly beneficial for merchant auditing and iterative model training. However, as this functionality fell outside the initial MVP scope, we deferred its implementation to prioritize core system requirements. Thus, it's also not shown within our database model for this very reason.

Architecture Diagram



The diagram above showcases our finalized system architecture of Subscriptions Flow AI. Within the diagram, it consists of a couple of different Tiers and Components with includes the following:

Frontend

This consists of our React framework application that Users would interact with. We chose React as it is one of the most widely used frameworks that everyone within the team has experienced developing full-stack applications in. Another reason for why we chose React was because it was one of the more complicated frameworks to work under the Shopify Domain, especially when it came to building an embedded application through their platform. On top of this, when we had initialized our application through Shopify's CLI tools, building an application using React seemed to be the most popular choice to make as then we could access a bunch of in-built libraries and packages from Shopify's end, with one of those specific libraries being Polaris UI. Polaris UI has a ton of pre-configured visual components that can easily be imported into the Shopify Domain. However, there were some situations where we had to create our own UI components from scratch using plain HTML/CSS due to the library having its own rigid constraints.

Backend

This consists of our Typescript server that handles all of the internal business logic of our application like our subscription management and customer feedback reporting workflows as an example. We chose Typescript for similar reasons as why we chose React as it's a programming language that the team has experience with, it's compatible under Shopify's Domain, and access easily a bunch of the built-in libraries and packages from Shopify's end. For the backend specifically, we used Shopify's own graphql apis to query the data that Shopify can natively handle and we needed to implement our internal logic (refer to the database model on what specific data we were retrieving from their end). We also utilized Shopify's cloudflare servers to locally run our own instances of the application within our

development stores for testing. Lastly, Shopify gave us a pre-configured instance of Prisma ORM when we initialized our application. This would then allow us to create our own data tables which would then be able to be hosted via the cloud (refer to the database model on what features we specifically made these custom data tables for).

Cloud

For cloud, we ended up choosing AWS as our main cloud provider, as the team has the most experience using this service compared to other providers like Azure and GCP. According to our research, it is widely used when connecting external components to the Shopify platform. In terms of our system architecture, it consists of three different parts: our hosting stack, our main database storage stack, and our customer feedback analytics stack.

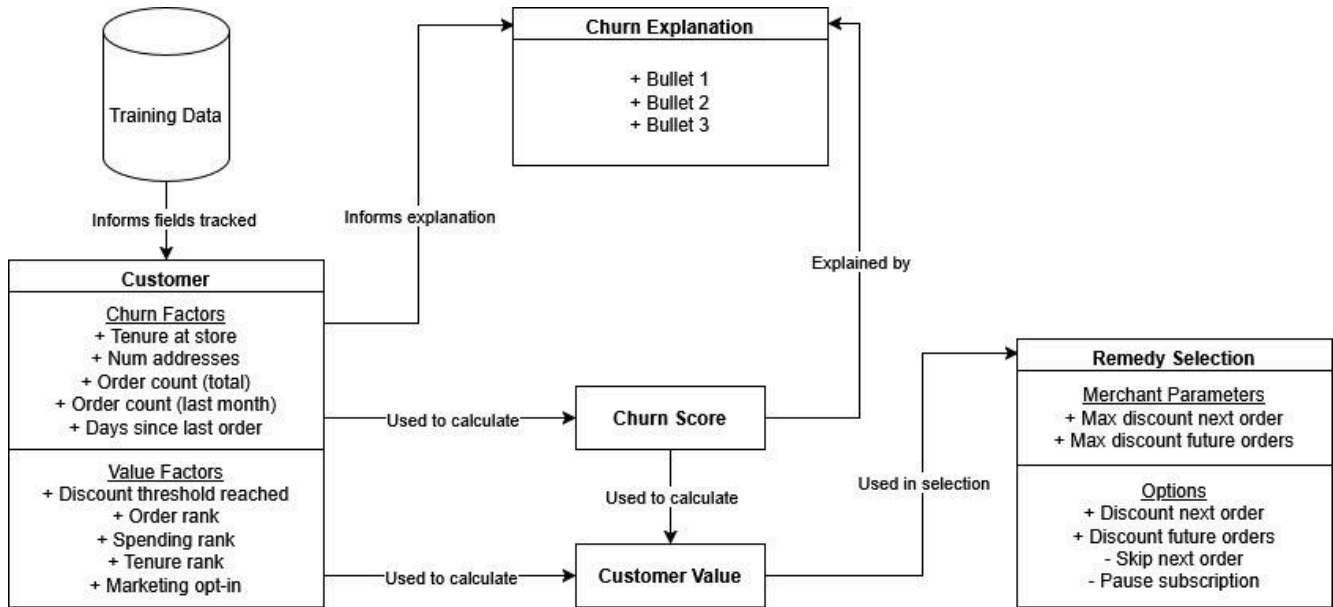
For our hosting stack, it's within a VPC container, consisting of our main EC2 Nginx reverse proxy server and an AWS Directory Service for identity management. Initially, we thought that Shopify had the capabilities to host our embedded application once it got approved, but it turns out the approval process itself required us to host the app via our own server first. After doing further research, we also found out that Shopify doesn't have the capability to host any third-party applications that aren't physically owned by them. Thus, we had to quickly pivot and host our application through the cloud. Given both the team's experience as well as the sponsor's input, we decided to implement a hosting stack given the services mentioned above.

For our storage stack, it's isolated within another dedicated VPC container, featuring an RDS PostgreSQL instance. This database manages our custom data tables, with the schema maintained and queried via Prisma ORM. To enforce the principle of least privilege, a security group firewall restricts network access exclusively to our backend service functions. We selected RDS because it supports complex analytical queries and provides the data integrity and consistency required for our subscription management application. Furthermore, we chose PostgreSQL to leverage our team's prior technical expertise, ensuring that we can streamline development and deployment on AWS.

For our customer feedback analytics stack, we implemented a daily automated workflow using EventBridge and AWS Lambda to improve the learning capabilities of our AI model to provide more accurate ChurnGuard reports. This pipeline retrieves customer feedback from RDS and reformats it into stylized spreadsheets, which are then stored in an S3 bucket. This serverless approach allows for background data processing without impacting the performance or resource availability of our primary application. As of now, the spreadsheets that are currently being created through this workflow aren't being ingested into our backend AI model due to time constraints, but due to our ability to at least deploy this working pipeline, we decided to showcase it within our system architecture.

Lastly, in regards to overall security within system architecture, it's all secured by HTTPS/TLS encryption to ensure any data-in-transit is protected from any interception attacks. This is a critical safeguard procedure to follow for any application operating on the public internet. Furthermore, these protocols are strictly enforced by both Shopify and AWS themselves as the standard cryptographic protocol for all communication between our backend and the Shopify API, as well as for authenticating requests to our cloud infrastructure through IAM roles.

AI



The first step of ChurnGuard is the selection of factors that are known to affect churn. This is done by inputting a spreadsheet of compiled training data into a logistic regression model powered by SciKit Learn. We started with an existing spreadsheet where each row contains whether a customer churned along with various factors associated with the customer. As more users continue to use Subscriptions Flow AI, the number of factors we can use to calculate churn score will increase. Though the logistic regression model will determine weights for each churn factor, it requires a lot of data to be accurate (the original spreadsheet had 5000+ rows). As such, substantial use of the app will be necessary before new churn factors can be added.

Currently, all customer churn factors and value factors can be retrieved using Shopify APIs. Customer value factors, along with the churn score, are used to determine the amount of resources worth expending (ie. discount given) to retain a customer. This calculation considers how much a customer contributes to a merchant's store with how likely they are to cancel. This "customer value" determines which option will be presented to the customer the next time they attempt to cancel a subscription.

Lastly, the process for obtaining a churn explanation is to send a prompt containing a churn score and associated churn factors to the Gemini API, the response from which is parsed into a summary consisting of three bullet points. Ideally, the churn explanations would be stored in their own type of report for accountability purposes, but this feature was completed right on the edge of the code freeze and thus could not be expanded.

Process and Product Metrics

Our team employed several process and product metrics throughout the project, including sprint velocity, effort by type of activity, SUMI, and Core Web Vitals. These metrics allowed us to evaluate how effectively the team was working and whether the product was improving from the user's perspective.

For sprint velocity, the team tracked the number of story points completed during each sprint. The recorded sprint velocities were 22, 3, 37, 66, 55, 52, 25, and 29, with an average velocity of about 36 points per sprint. The results show that the team's productivity increased after the early sprints, especially during Sprints 4 through 6. The higher velocity during this period was largely because the team was implementing the bulk of the project's features in an effort to complete the MVP before the semester ended. During this time, all team members contributed more than 12 hours of work per week, which helped the team make significant progress. However, the lower velocity in the final sprints indicates that later work involved more deployment, testing, documentation, integration, and refinement rather than only completing new feature work.

The team also tracked effort by type of activity using overall project data. The largest portion of effort was spent on development, which accounted for 49.9% of the team's work. This was expected because the main goal of the project was to build a functional MVP. Deployment accounted for 16.2% of the effort, which was also significant because the team hosted the application on its own AWS server and had to follow the Shopify review process. This required the team to carefully check the application against Shopify's required checklist before submission. Research accounted for 12.7% of the effort, largely because the team spent a considerable amount of time researching the Shopify API and understanding how to integrate it correctly. Design and documentation accounted for 11.8%, reflecting the time spent planning the user experience, documenting decisions, and preparing project materials. Finally, bug fixing and refactoring accounted for 9.4% of the effort, showing that the team also spent time improving code quality, resolving defects, and stabilizing the system as features were implemented.

For product usability, the team used SUMI to evaluate the user experience. The initial SUMI overall score was 65.1, which was moderately positive. A later SUMI result showed an improved overall score of 79.92. Several categories improved significantly, including efficiency, control, helpfulness, and learnability. This indicates that the system became easier to use and more effective after the team made improvements based on feedback. The SUMI results suggest that usability was one of the areas that improved most over the course of the project.

The team also considered Core Web Vitals as a performance and user experience metric. To support this, the team implemented a hook to collect Core Web Vitals data, including information related to page loading performance, responsiveness, and visual stability. However, no official Core Web Vitals data has been collected so far. Even though official results are not available, implementing the data collection hook was an important step toward evaluating front-end performance and improving the user experience in future iterations.

Overall, the metrics show that the team performed well in building the MVP, improving usability, and maintaining steady progress during the most important development period. The strongest results were the improvement in SUMI scores and the high development output during Sprints 4 through 6, when the team worked heavily to try to complete the MVP before the semester ended. The effort data also shows that the team successfully focused most of its time on development while still dedicating meaningful effort to deployment, Shopify API research, design, documentation, and code quality. The weaker areas were the inconsistency in sprint velocity and the additional time required for deployment due to AWS hosting responsibilities and the Shopify review process. These results suggest that the team made strong progress but also faced challenges with estimation, deployment complexity, platform requirements, and late-stage project refinement.

Product State at Time of Delivery

The final state of the product is a complete Minimum Viable Product hosted on AWS that supports a full subscription management workflow as well as providing churn interventions, insights, and merchant customization of selling plan groups. The product is currently going under review on Shopify for listing approval, and has been overdue a response for nearly a month.

In terms of missed features, it's crucial to note that every feature that was in the original MVP has been accounted for in some way. The major missing feature from the MVP is email templates, and the ability for merchants to customize specific templates to be sent out automatically based on certain conditions, such as a failed transaction, no inventory, and delayed delivery to name a few. The team was able to develop most of this, but when it came to setting up the inbox, Shopify didn't provide its own customizable mailing service, and many of those mentioned emails, such as failed transactions, are already done automatically. Given the short timespan of the project it was deemed unnecessary for the MVP.

The other few features are relatively small, one being free shipping, and the other being additional bonus features that were discussed, but were knowingly unlikely to make the cut. For free shipping all the components were created and data was properly recorded for subscriptions, but the way Shopify handles free shipping is mainly through discount codes which was unexpected. For the MVP it was also deemed unnecessary and other parts of the projects should be focused on. None of the bonus features had time to be developed, but to name a few there were migrations and a win-back feature. Migrations included having a setup process if users had already used a different subscription management app and were coming over. The win-back feature would send out an email to a cancelled customer after some time in an attempt to win them back. Overall these features were deemed unnecessary for the deployment of the MVP, and could be easily added as the project evolves.

Our main focus especially in the final semester was to ensure that our MVP was as polished as possible to have the best chance of approval with Shopify. Unfortunately Shopify has responded saying that they are incredibly behind and have a large backlog of approvals to go through, meaning our timeline was pushed back in a way that was out of our control. In the meantime though, the team was able to add more polish to the product as a whole to ensure it was something we could be proud of.

Project Reflection

This project has been an incredible experience all around. The entire team was jumping into a domain that none of us had too much experience in, with a relatively unfamiliar techstack. But through that we persevered and grew leaps and bounds as a team. In reference to Tuckman's Stages of Group Development, the team did not have a storming phase amongst each other, from the day we all came together, we were incredibly cohesive. Each of us had our strong suits and we played to them perfectly, whether that be design, communication, or a specific aspect of development. We allowed each other to focus on their strong-suits while lifting others in similar fashions if they lacked experience. The team was also incredibly communicative of issues and ideas during meetings, which led to continual improvement as a group.

One of the major pain points during the project was consistent sponsor communication outside of meetings. Due to how quickly our project was evolving, especially since this project started from scratch, a lot of the time communication was needed outside of our weekly meetings. The first

semester the need wasn't as prevalent due to us getting used to the domain still and fleshing out features, but near the end where specificity mattered most, we shifted most sponsor communication to our own Discord server, where we were able to communicate whenever it was needed. Another side of team communication we improved over time was during team meetings, namely having efficient meetings. Our meetings would go on for 2 and a half to 3 hours consistently during the first semester, but most of them were a subset of the team discussing a certain task, while the rest of the team had little input. We decided to cut down this time, and ensure that all communication during team-wide meetings incorporated the entire team as much as possible to not waste anyone's time, and shift smaller conversations either after the meetings or online. This shift alone improved our time efficiency dramatically and forced us to hone in on the most important topics during meetings.

One of the issues we were consistently faced with was an unknown scope scale. It was difficult to properly gauge task sizes due to our lack of knowledge on the topic/task. This obviously gradually improved, but when starting a new part of the project such as setting up the app for deployment, it was once again stepping into a new world. To combat this we tended to oversize these tasks, and as we gained a better understanding, would create subtasks or new stories when necessary. But there were so many varying elements to this project that at some point it did feel like we were just shooting in the dark again and again for tasks such as getting AI churn reports, hosting the app, meeting Shopify requirements, etc.

If we were to restart this project, there would be 3 major differences. First off, sponsor communication would immediately be moved to Discord. The consistent communication we had near the end of the project was such a huge improvement, and would have saved us the headaches of not being able to see our sponsor responses due to their email being blocked by the school. Secondly, we would have structured our project slightly differently. Overall we weren't too far off, but if we knew exactly what we knew before we started our project, we would have been able to leverage reuse far more efficiently, and not have rewritten similar code to another developer, especially at the start of the project. Finally, we would know not to rely on Shopify too much and potentially have submitted our app for review far before it was ready to be listed. It's unfortunate that we won't be able to see (at least during a time that it matters) how our app fares on a trusted 3rd party evaluation system. Even if it were to be rejected, we would have gotten some crucial feedback, but unfortunately the ending point of our project will be in sort of a lull, just waiting for a response.

Overall this project has been a one-in-a-lifetime learning experience for us all. We were incredibly lucky to have such a dedicated team with a great sponsor and coach who constantly supported us. There were plenty of times where we felt like we were potentially in over our heads, and also times where we thought that we could get it done easily, but nevertheless the team always pulled together when it was needed most, and truly made a project we can all be proud of.

References

1. Shopify App Approval Process: <https://shopify.dev/docs/apps/launch>
2. Shopify GraphQL API Documentation: <https://shopify.dev/docs/api/admin-graphql/latest>
3. Shopify Polaris UI Documentation: <https://shopify.dev/docs/api/polaris>